

Project and Process Metric

Project Metric 1: Filter Update Response Time

What: Elapsed time for the CECS event feed and embedded map pins to update after a user applies a filter combination (time, location, cost, cultural credit, tags).

How: Run automated integration tests that simulate worst-case filter combinations on a populated test database. Capture total update time in milliseconds (start: user submits filters; end: feed + map render complete).

Why: Directly tracks performance against the CECS performance target (1–2 seconds) and gives early warning for the “Filter Lag” risk (R1) and map-render coupling risk (R3).

Project Metric 2: Actual vs. Budgeted Effort

What: Difference between estimated person-hours and actual person-hours spent refactoring EventService and implementing the Strategy Pattern for filtering.

How: Each team member logs time per refactoring task card (e.g., Discord or a shared spreadsheet). Weekly totals are compared against the original estimate. Report as: Effort Variance = Actual – Budgeted (hours) and % Variance.

Why: Shows schedule slip in real time. If refactoring consumes more effort than planned, the team can cut scope or re-sequence work before it blocks downstream integration (like map changes and approval workflow).

Process Metric 1: Defect Removal Efficiency

What: How well the team’s tests catch errors before users see them. $DRE = E / (E + D)$, where E = errors found before delivery, D = defects found after delivery.

How: Track E from Formal Technical Reviews (FTRs) and Continuous Integration regression test failures (count each confirmed issue once). Track D from post-release bug reports (students/staff) and production logs. Compute DRE per milestone and trend it over time.

Why: A strategic quality signal. DRE near 1.0 means the process filters out most errors before release. Low DRE means bugs are escaping into the hands of students/staff, which increases rework and weakens trust in cultural credit tracking.

Process Metric 2: Error Density

What: Number of errors found per unit size of a work product. Example: errors per KLOC (thousand lines of code) or errors per reviewed UML artifact.

How: During code reviews, count confirmed issues per module (e.g., MapService, ApprovalService, EventService). Divide by module size (KLOC) for code or by number of diagram elements for models. Track as a time series per module.

Why: Detects chronic trouble spots. If MapService shows higher error density than other modules, the team can allocate more review time, add targeted tests, or improve documentation/training for that area in future work.